Miniproject 2 report

Mohammadamin Alibakhshi
Course 5743

1 Project Objective

In this mini project, We are given a 5-class multi-label chest X-ray dataset and the task is to design a classifier in PyTorch. Each image can have multiple positive labels among `class1, class2, class3, class4, class5`. The main goal is to improve macro F1 while keeping the checker-compatible train/test pipeline.

2 Model and Training Setup

2.1 DenseNet121 Backbone

I used a pretrained `DenseNet121` model from `torchvision.models` [2]. The final classifier is replaced with a small head: Dropout($p = 0.3$) + Linear layer with 5 outputs.

2.2 How We Update Weights and Learning Rate

The training uses two stages:

- **Epoch 0 (first epoch):** backbone (`model.features`) is frozen. Only the classifier head is updated.
- **From epoch 1 onward:** backbone is unfrozen and the whole network is fine-tuned.

For optimization:

- Stage 1 uses learning rate $LR = 0.001$.
- Stage 2 uses lower learning rate $LR \times 0.2 = 0.0002$, so pretrained weights are updated more carefully.

Loss function is `BCEWithLogitsLoss`. (binary cross-entropy loss + sigmoid in one function) In this loss function, mistakes on positive labels receive an extra penalty, which is especially helpful for rare classes.

Early Stopping: Because we save the model after each epoch in which the validation performance improved, I sometimes shut down the network manually rather than using early stopping. However, there is an implementation of simple early stopping on validation macro F1 with patience = 3 epochs.

3 Preprocessing and Data Corrections

The input preprocessing is:

- Read X-ray as grayscale
- Resize to 320×320 (Even though the Densenet121 uses 224, I used for more resolution)
- Convert to float and scale to $[0, 1]$ by dividing by 255
- Replicate grayscale to 3 channels (because the Densenet receives three-channel images).
- Normalize with ImageNet mean/std [3]

4 Augmentation Strategy

I used a scarcity-aware but more diverse augmentation policy. For each training sample, 2 or 3 operations are randomly selected from:

- affine (rotation, translation, scaling)
- tone transforms (gamma or brightness/contrast)
- CLAHE : CLAHE is a local contrast enhancement method. It improves visibility in darker or lower-contrast regions by enhancing contrast within small local areas of the image rather than applying one global adjustment to the whole image
- Gaussian noise and optional blur
- cutout

This was designed to reduce naive minority-only near-duplicate repetition.

4.1 MixUp (Active)

As another augmentation strategy, MixUp is active in training: with a probability of 0.30, two images and their multi-label targets are linearly mixed. For our multi-label tasks, this can regularize the model and reduce overconfident fitting by using soft target combinations. This helped my model and increased the F1 score by 0.05 when it was saturated.

4.2 CutMix (Experiment Note)

CutMix was tested conceptually, but in this setting it did not work well, so it is not part of the current active final training code.

5 Threshold Strategy

After each epoch, the validation logits are passed through a sigmoid function to obtain probabilities. Then, a separate threshold for each class is tuned using the validation set. In this setup, the loss function is used during training, while the sigmoid and threshold tuning step is used afterward to convert predicted probabilities into final class labels based on the validation data.

These thresholds control the final binary decision boundary per class, which directly affects precision/recall balance and macro F1.

The best model is selected by validation macro F1, and tuned thresholds are saved in a text file. In inference (`test_Ersa.py`), thresholds are loaded from file with fallback values if the text file is missing. In the case of competition, I manually enter the thresholds to fallback values into the `test_Ersa.py` each time before submitting.

6 Architecture Figure

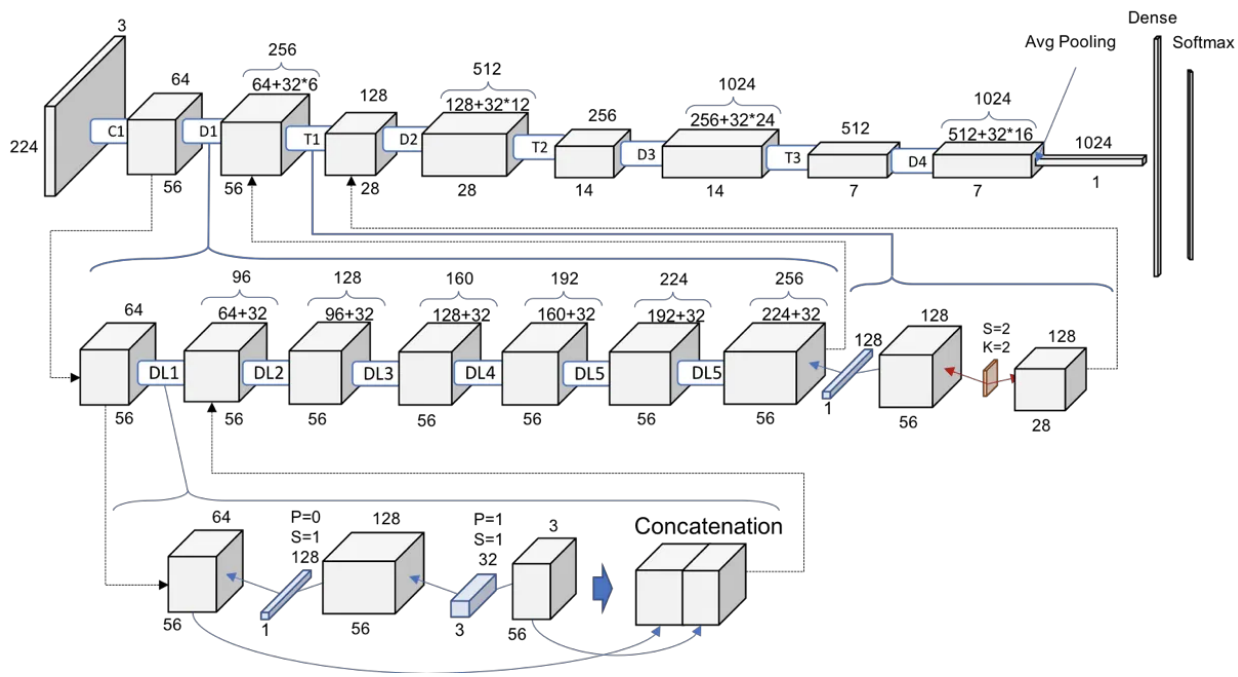

Figure 1: Pretrained DenseNet121 architecture illustration for conceptual reference. This image is not consistent with our class representation of a neural network. Numbers above the layers may change because the image input size is 320 in our case [1].

7 Notes

I tried training the network at 15%, reducing the validation split to 2%, and adding more data to the train loop, but it didn't make a significant difference. Thus, I chose 15%.

A strategy that helped me in both this project and Mini Project 1 was looking at the F1 score for each class separately. That made it easier for me to see where the network was lacking confidence and where it might be overfitting. Since we are working with imbalanced training data, like classes 4 and 5 in this project, we usually need to either augment those classes more or shift the decision boundary a bit in their favor. But in both cases, there is always a risk of overfitting. So the main challenge is finding the right balance between compensating for the limited data in those classes and not pushing the model into overfitting.

References

[1] DeepKapha Notes. Implementing densenet-121 in pytorch: A step-by-step guide. `https://medium.com/deepkapha-notes/implementing-densenet-121-in-pytorch-a-step-by-step-guide-c0c2625c2a60`, n.d. Medium article, accessed: 2026-04-10.

[2] PyTorch Contributors. Densenet121. `https://docs.pytorch.org/vision/main/models/generated/torchvision.models.densenet121.html`, 2026. Accessed: 2026-04-10.

[3] Stack Overflow Community. Why pytorch officially use mean = [0.485, 0.456, 0.406] and std = [0.229, 0.224, 0.225]? `https://stackoverflow.com/questions/58151507/why-pytorch-officially-use-mean-0-485-0-456-0-406-and-std-0-229-0-224-0-2`, 2019. Accessed: 2026-04-10.